DOCUMENTO TÉCNICO · HANDOFF

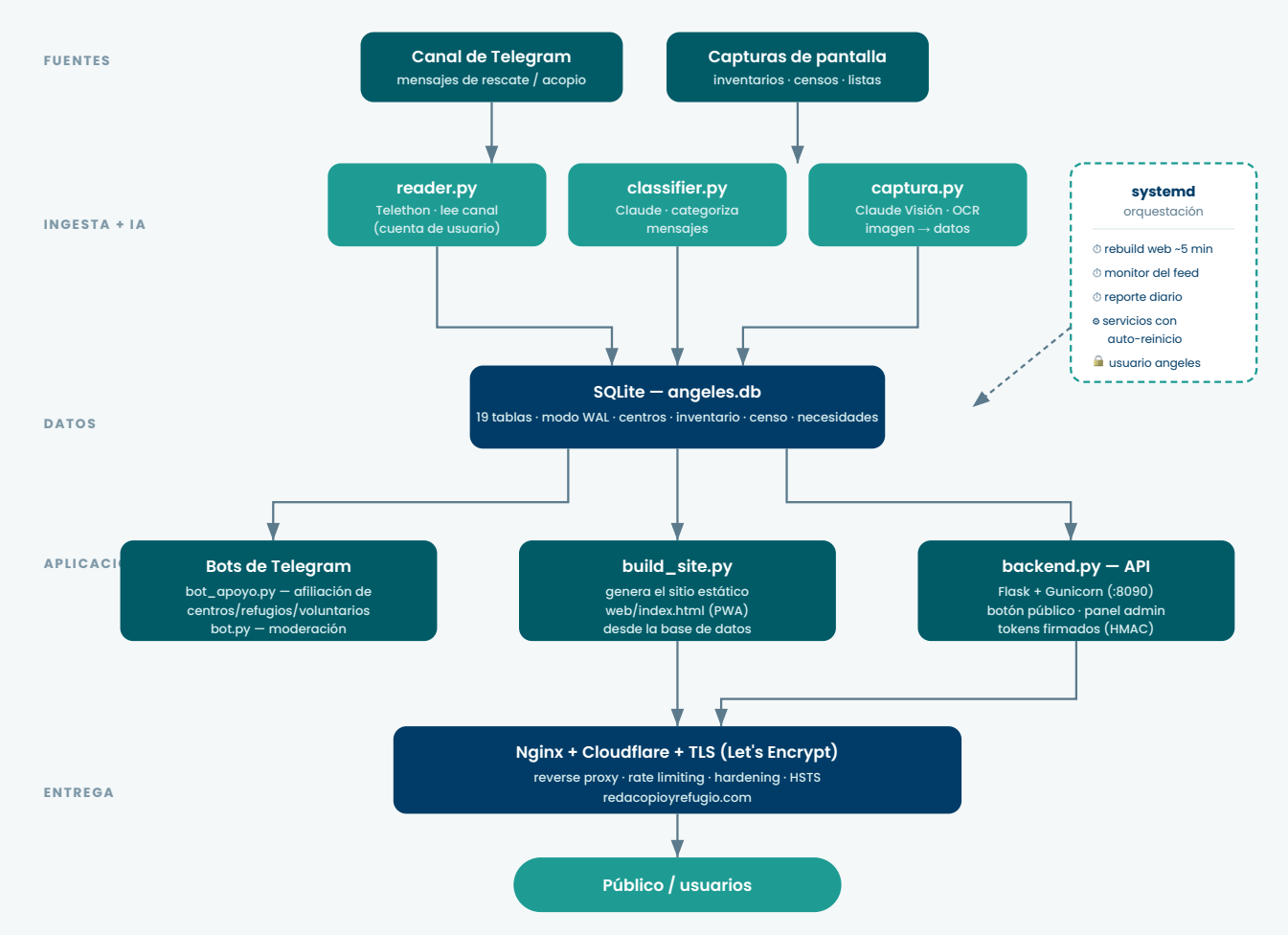
Red de Acopio y Refugio

Descripción técnica de la plataforma · para empresa desarrolladora

Producción: redacopioyrefugio.com (HTTPS activo) • **Infra:** VPS Ubuntu (un servidor) • **Lenguaje:** Python 3 (venv), ~6.400 LOC propias • **Base de datos:** SQLite WAL (~2,1 MB, 19 tablas)

Resumen. Plataforma de coordinación de ayuda humanitaria (acopio, refugios y rescate) construida sobre un canal de Telegram, con inteligencia artificial (Claude) para estructurar la información y una web pública en vivo. Funcionando en producción con datos reales.

1. Arquitectura general



2. Componentes principales

Pieza	Archivo(s)	Función
Ingesta + IA	`reader.py`, `classifier.py`, `captura.py`	Lee el canal de Telegram, clasifica y extrae datos con IA (Claude)
Bot de afiliación	`bot_apoyo.py` (2.503 líneas)	Bot conversacional donde centros/refugios/voluntarios se registran y gestionan inventario
Bot de moderación	`bot.py`	Bot secundario de moderación del canal
Web + API	`build_site.py` (1.604 líneas), `backend.py`	Genera el sitio público estático y expone la API del botón público

3. Stack tecnológico

- **Telethon 1.36** — cliente de Telegram (cuenta de usuario) que lee el canal de origen.
- **API Bot de Telegram** (vía `urllib`, long-polling `getUpdates` / `sendMessage`) — bots de afiliación y moderación.
- **Anthropic Claude** — clasifica mensajes y hace **visión por computadora**: lee capturas (inventarios, censos) y devuelve datos estructurados (Pydantic).
- **Flask + Gunicorn** (3 workers, `127.0.0.1:8090`) — backend de la API.
- **Pillow** — generación de iconos PWA y procesamiento de imágenes.
- **Nginx** — reverse proxy, TLS y servidor de estáticos. **systemd** — servicios y tareas programadas.

4. Flujo de datos (ingesta con IA)

1 Telethon lee mensajes nuevos del canal. 2 `classifier.py` envía cada mensaje a Claude → lo categoriza (heridos/hospitales, donaciones, rescate, alertas, contactos). 3 `captura.py` procesa capturas con visión IA → extrae inventario y censos. 4 Todo se guarda en SQLite. 5 `build_site.py` regenera la web. 6 Un **timer de systemd corre el ciclo cada ~5 minutos**.

5. Modelo de datos (19 tablas · datos reales)

Tabla	Filas	Para qué
`mensajes`	3.685	Mensajes crudos del canal
`inventario`	2.113	Ítems de insumos (categoría, cantidad, detalle)
`centros`	217	Puntos de acopio/refugio con estado (disponible/lleno)

`capturas`	75	Imágenes procesadas con IA
`servicios`	56	Servicios/contactos detectados
`llamados`	7	Llamados de auxilio (S.O.S. con foto y sector)
`representantes`	6	Responsables de cada punto (geolocalización, aforo, baños, camas, agua, electricidad…)
`voluntarios`, `aliados`, `necesidades`, `solicitudes`, `censo_personas` …	—	Voluntariado, aliados comerciales y motor de emparejamiento oferta/demanda

Motor de emparejamiento: un punto declara qué le falta (`necesidades`) y el sistema lo cruza con otro punto que tiene ese insumo (`solicitudes`), con estados `buscando → disponible → coordinando → resuelta` .

6. El bot de afiliación (núcleo funcional)

`bot_apoyo.py` es la pieza más grande. Maneja:

- **Registro por rol:** representante de acopio / hospital / clínica / refugio, voluntariado, "manos solidarias" (aliados).
- **Ficha del inmueble:** aforo, camas, duchas, cocina, agua potable, electricidad, planta eléctrica, accesibilidad, geolocalización.
- **Inventario con 3 estados** y depuración reversible.
- **Flujo de aprobación** (pendiente → aprobado/rechazado) con panel de administración.
- **Solicitudes entre puntos** y notificaciones.

7. Web pública y API

- **Sitio estático** generado server-side, servido por Nginx con caché. Es una **PWA** (iconos 192/512, apple-touch-icon).
- **API Flask** bajo `/api/`: health-check, `datos.json`, panel admin con autenticación, exportación maestra, registro de visitas con **tokens firmados (HMAC + salt)** y endpoint de reportes.

8. Seguridad e infraestructura

- **TLS** con Let's Encrypt (renovación automática, redirección 80→443, HSTS).
- **Cloudflare** delante; Nginx valida el origen y bloquea acceso directo.
- **Hardening Nginx:** rate limiting (`limit_req` / `limit_conn`), bloqueo de archivos sensibles (`.env` , `.db` , `.py` , `.bak`), límites de body/timeout, cabeceras de seguridad.
- **Aislamiento:** corre bajo usuario dedicado `angeles` (no root); secretos en `.env` con permisos 600.
- **systemd** (auto-reinicio): `angeles-web` (gunicorn), `angeles-apoyo` (bot afiliación), `angeles-bot` (moderación), más timers de rebuild, monitor del feed y reporte diario por Telegram.

9. Estado actual y consideraciones para el handoff

Funcionando en producción con datos reales (3.685 mensajes, 217 centros, 2.113 ítems de inventario). Deuda técnica típica de un MVP de un solo autor que la desarrolladora debe conocer:

Aspecto	Situación
Estructura del código	Módulos monolíticos grandes (`bot_apoyo.py` y `build_site.py` superan 1.500–2.500 líneas).
Control de versiones	Sin Git ni CI/CD; respaldos como copias `.bak` con timestamp en el mismo servidor.

Base de datos	SQLite de archivo único: bien para el volumen actual, limita concurrencia y multi-servidor.
Testing / entornos	Sin suite de tests automatizada; staging solo parcial (sandbox `angeles-sbx`).
Despliegue	Manual (edición directa en el VPS), no contenerizado.